Ignytion io

IGNYTION IO · TECHNICAL LAB NOTE

Python for VLSI Engineers
The Complete Lab Manual

Twenty four reproducible experiments, from a Crucible workspace to an AI ready context packet.

Twenty four hands on experiments covering workspace automation, RTL generation, verification with cocotb and pyuvm, Yosys synthesis, quality of results analytics, physical design metrics, and AI ready context generation. Every command in this manual was run as printed and every figure is captured output. No reconstructed logs.

SECTION	EXPERIMENTS	CLASSES	VERIFIED
01 Foundations	01 to 09	R	✓ 9 / 9
02 Verification and Waveforms	10 to 15	B, R/D	✓ 6 / 6
03 Synthesis and Quality of Results	16 to 20	B, R/D, R	✓ 5 / 5
04 Physical Design and AI Context	21 to 24	D, R/D, R/B, R	✓ 4 / 4

Class codes. **R** is pure Python, or a pip installable dependency. **D** is dataset backed, running on bundled sample artifacts. **B** needs an external EDA binary, and detects it before invoking it. A slash means both apply to the same experiment: **R / D** runs today on bundled data and would also run live, and **R / B** reads its data through a pip module while a second capability stays gated behind the binary. Page 03 expands each one.

CONTENTS

What this manual covers

01	Introduction and how to read this manual	3
02	Suite overview, experiments 01 to 12	4
03	Suite overview, experiments 13 to 24	5
04	Section 1 · Foundations	6
05	Section 2 · Verification and Waveforms	24
06	Section 3 · Synthesis and Quality of Results	36
07	Section 4 · Physical Design and AI Context	46
08	Closing summary	54

INTRODUCTION

Python is the control plane. Crucible is the reproducibility layer.

Each experiment in this manual produces a real, reproducible artifact: a workspace manifest, a synthesised netlist, a cocotb regression, a timing summary, an AI ready context packet. Python automates the chip design workflow; Crucible makes every run of it reproducible on another machine.

REPRODUCIBILITY CLASSES

CLASS	WHAT IT MEANS	RUNS TODAY
R	Pure Python, or a pip installable dependency	✓ **Anywhere**
D	Dataset backed, using bundled sample artifacts	✓ **Anywhere**
B	Needs an external EDA binary, detected first	**Gated, cleanly**

HOW TO READ AN EXPERIMENT

Every experiment spans two pages. The first sets up the aim, the theory, the apparatus, and the exact procedure, closing with the core logic that makes the result trustworthy. The second carries the captured evidence from a live run, states the result, and closes with a card naming the single number that run produced.

> **A class B experiment reporting GATED has not failed**
>
> Four experiments need an external binary. Each one detects its tool before invoking it and writes a GATED status when the tool is absent, which is the correct result on a machine without it. Every console block on an outcome page is real output, captured while verifying this manual.

SUITE OVERVIEW

All twenty four experiments, 01 to 12

Class marks how an experiment runs today. The page column points at its setup page.

NO.	EXPERIMENT	CLASS	PAGE
01	Entering a Crucible Python-for-VLSI Workspace	R	06
02	Semiconductor File and Folder Automation	R	08
03	Data Structures for RTL Metadata	R	10
04	Read and Write VLSI Artifact Files	R	12
05	Parsing Verilog Module Headers	R	14
06	Test Vectors for a Combinational DUT	R	16
07	ALU Golden Model	R	18
08	Template-Based RTL Generation	R	20
09	Register Map from YAML	R	22
10	Automating Verilator Simulation	B	24
11	Parsing Simulation Logs	R / D	26
12	First cocotb Testbench	B	28

SUITE OVERVIEW

All twenty four experiments, 13 to 24

NO.	EXPERIMENT	CLASS	PAGE
13	Reusable cocotb Driver and Monitor	B	30
14	VCD Waveform Parsing	R / D	32
15	GTKWave Launch Helper	B	34
16	Running Yosys from Python	B	36
17	Parsing Yosys Reports	R / D	38
18	Yosys JSON Netlist Analysis	R / D	40
19	QoR Comparison with pandas	R	42
20	Plotting QoR Trends	R	44
21	OpenLane / OpenROAD Metrics Collection	D	46
22	Timing Report Parser	R / D	48
23	KLayout Helper Automation	R / B	50
24	AI-Ready Context Packet Generation	R	52

SECTION 1 · FOUNDATIONS / EXPERIMENT 01 · CLASS R

Entering a Crucible Python-for-VLSI Workspace

AIM

To enter a reproducible Python-for-VLSI workspace, inspect the managed Python runtime, verify the standard folder structure, and produce a workspace manifest artifact that records the runtime and layout state.

THEORY

A workspace differs from a random folder in that its state is recorded: the runtime version, the tool bindings, and the directory layout are all written down where a later run can check them.

APPARATUS

```
Crucible workspace (igny)
Python 3.9 or newer, stdlib only
No EDA binaries required
```

ARTIFACT CONTRACT

```
Report state, never repair it.
Only reports/ is created,
because the script must write
the manifest there.
```

PROCEDURE

```
cd experiments/lab01_workspace_entry
python workspace_manifest.py
igny run script ./workspace_manifest.py
```

CHECK FOLDERS

```
EXPECTED_DIRS = ["rtl", "tests",
  "scripts", "generated", "reports"]
def check_folders(root):
  return {n: (root/n).is_dir()
          for n in EXPECTED_DIRS}
```

GUARANTEE

```
Never calls mkdir on an
expected folder. A green
checklist would mean the
checker is lying.
```

EXPERIMENT 01 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
IGNYTION CRUCIBLE - WORKSPACE MANIFEST
========================================
Python version    : 3.14.5
Python executable : C:/Python314/python.exe
Platform          : Windows-11-10.0.26200-SP0

Expected workspace folders:
  [ ] rtl          MISSING
  [ ] tests        MISSING
  [x] reports      present

Missing folders   : rtl, tests, scripts, generated
```

RESULT

The run captured the Python version, the interpreter path, and the platform, then audited the folder layout. It reported the absent folders rather than creating them, which is what keeps an audit honest.

✓ **Experiment 1 · PASS**

Key result: Runtime fingerprint captured; folder audit correct on a bare workspace.

SECTION 1 · FOUNDATIONS / EXPERIMENT 02 · CLASS R

Semiconductor File and Folder Automation

AIM

To write a reusable Python script that generates a standard VLSI block project skeleton for a new block, so every IP block begins from an identical, reproducible directory structure.

THEORY

Chip projects share deeply repeated structure. When engineers create it by hand the layout drifts, one person's `tb/` becomes another's `tests/`. A scaffolder fixes this once, for every future block.

APPARATUS

```
Python 3.9 or newer
argparse, pathlib
No EDA binaries required
```

LAYOUT CHOICE

```
Block first: <block>/rtl/,
not family first: rtl/<block>/.
One block, one folder, one
unit to archive or delete.
```

PROCEDURE

```
cd experiments/lab02_folder_automation
python scripts/new_block.py counter8
python scripts/new_block.py counter8 --force
```

NON-DESTRUCTIVE CREATE

```
def create_block(block, *, force=False):
  for d in block_dirs(root, block):
    state = ("recreated" if force
      else "exists") if d.exists()
      else "created"
    d.mkdir(parents=True, exist_ok=True)
```

GUARANTEE

```
mkdir never deletes.
A --force run only
re-ensures directories,
files are always kept.
```

EXPERIMENT 02 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Block skeleton for 'counter8':
  [  created] counter8/rtl
  [  created] counter8/tests
  [  created] counter8/reports
  [  created] counter8/generated

usage: new_block.py [-h] [--force] block
new_block.py: error: the following arguments are required: block
```

RESULT

A reusable, idempotent, non-destructive scaffolding script was implemented. It generates a self-contained four-folder subtree per block, seeds an intent README, and preserves existing source on every re-run.

✓ **Experiment 2 · PASS**

Key result: Idempotent scaffolder verified across create, re-run, and force.

SECTION 1 · FOUNDATIONS / EXPERIMENT 03 · CLASS R

Data Structures for RTL Metadata

AIM

To represent an RTL module's interface, its name, parameters, ports, and clock or reset assumptions, as structured Python data, and export it as JSON for downstream generators to consume.

THEORY

An RTL block carries metadata as well as source: its name, parameters, ports, and reset style. Left as free text inside a .v file no tool can read it. Python's `dict` and `list` map onto it directly.

APPARATUS

```
Python 3.9 or newer, stdlib
json, dataclasses
No EDA binaries required
```

VALIDATION RULE

```
A module record without a
clock or reset field is
rejected before export,
not discovered downstream.
```

PROCEDURE

```
cd experiments/lab03_rtl_metadata
python rtl_metadata.py
cat reports/metadata/counter8_metadata.json
```

PORT RECORD

```
def make_port(name, direction,
              width=1, desc=""):
  if direction not in
     {"input","output","inout"}:
    raise ValueError(...)
```

CHECK

```
Validation : OK printed
only when clock and reset
are both present in the
exported metadata record.
```

EXPERIMENT 03 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Module     : counter8
Ports      : 4
Inputs     : clk, rst_n, en
Outputs    : count
Validation : OK

{
  "module": "counter8",
  "clock": "clk", "reset": "rst_n",
  "summary": {"num_ports": 4}
}
```

RESULT

RTL module metadata was modelled with Python collections, validated for clock and reset presence, and exported as machine-readable JSON ready for a generator to consume.

✓ **Experiment 3 · PASS**

Key result: 4-port counter8 record validated and exported to JSON.

SECTION 1 · FOUNDATIONS / EXPERIMENT 04 · CLASS R

Read and Write VLSI Artifact Files

AIM

To read a sample simulation log file and produce a clean summary artifact in both human-readable and machine-readable form, using safe file input and output throughout.

THEORY

VLSI workflows are artifact-heavy. Two disciplines matter: safe I/O, a context manager closes the file even on error, and a clear distinction between the raw log and the trustworthy summary of it.

APPARATUS

```
Python 3.9 or newer, stdlib
re, json, pathlib
Bundled datasets/sample_sim.log
```

MISSING FILE RULE

```
FileNotFoundError raised
with the exact path, not a
cryptic traceback deep
inside the parser.
```

PROCEDURE

```
cd experiments/lab04_artifact_files
python artifact_io.py
cat reports/artifacts/log_summary.txt
```

SAFE READ

```
def read_log(path):
  if not path.exists():
    raise FileNotFoundError(
      f"log not found: {path}")
  with path.open() as fh:
    return [l.rstrip() for l in fh]
```

GUARANTEE

```
with open(...) closes the
handle even when the loop
raises partway through
the file.
```

EXPERIMENT 04 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
SIMULATION LOG SUMMARY
======================
Status        : PASS
Lines read    : 8
Warnings      : 2
Errors        : 0

Warnings:
  - signal 'dbg_unused' never toggled
  - clock jitter model disabled
```

RESULT

A simulation log was read safely and summarised into both TXT and JSON artifacts, with a missing-file guard and an errors-force-fail rule applied consistently.

✓ **Experiment 4 · PASS**

Key result: 8 lines read, 2 warnings and 0 errors summarised correctly.

SECTION 1 · FOUNDATIONS / EXPERIMENT 05 · CLASS R

Parsing Verilog Module Headers

AIM

To write a Python parser that extracts a module's name and port list, direction, width, and name, from a simple Verilog declaration, and export the interface as JSON and CSV.

THEORY

Engineers routinely need a module's interface for documentation, wrappers, and testbench skeletons. For simple ANSI-style headers, a regular expression extracts it far cheaper than reading by eye.

APPARATUS

```
Python 3.9 or newer, stdlib
re, csv, json
Bundled rtl/counter8.v
```

PARSE STAGES

1. Strip block and line comments first.
2. Match direction, width, and name per port.

PROCEDURE

```
cd experiments/lab05_verilog_parse
python parse_ports.py
cat reports/interfaces/counter8_ports.json
```

PORT PATTERN

```
PORT_RE = re.compile(
  r"\b(input|output|inout)\b"
  r"(?:\[(\d+):(\d+)\])?"
  r"\s+(\w+)")
```

LIMIT

```
A regex parser is exact
for ANSI headers only, not
for a general SystemVerilog
grammar.
```

EXPERIMENT 05 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Module: counter8
  input  [ 1b] clk
  input  [ 1b] rst_n
  input  [ 1b] en
  output [ 8b] count
```

RESULT

The module interface was extracted from Verilog source and exported to JSON and CSV, with comments correctly ignored and multi-bit widths computed from the bracket range.

✓ **Experiment 5 · PASS**

Key result: 4 ports extracted from counter8.v; widths computed correctly.

SECTION 1 · FOUNDATIONS / EXPERIMENT 06 · CLASS R

Test Vectors for a Combinational DUT

AIM

To generate directed and random test vectors for a parameterisable N-bit adder, each carrying a golden expected output from an independent Python model, with a fixed seed for reproducibility.

THEORY

Directed vectors target corner cases a human knows are risky: zero, maximum, and the overflow boundary. Random vectors broaden coverage, but only a **fixed seed** lets a failing run be replayed exactly.

APPARATUS

```
Python 3.9 or newer, stdlib
random, csv, argparse
No EDA binaries required
```

GOLDEN MODEL

```
mask = (1 << width) - 1
sum = (a + b) & mask
carry = (a + b) >> width
  & 1
```

PROCEDURE

```
cd experiments/lab06_test_vectors
python gen_vectors.py --width 4
  --num-tests 20 --seed 1
```

RANDOM VECTORS

```
def random_vectors(width, n, seed):
  rng = random.Random(seed)
  maxv = (1 << width) - 1
  return [(rng.randint(0, maxv),
    rng.randint(0, maxv))
    for _ in range(n)]
```

GUARANTEE

```
A local Random(seed) never
shares state with any
other call in the process.
```

EXPERIMENT 06 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
width=4 seed=1
vectors: 26 (6 directed + 20 random)

idx,a,b,sum,carry_out,kind
0,0,0,0,0,directed
3,15,15,14,1,directed
```

RESULT

A reproducible set of directed and random adder test vectors with golden outputs was generated and written to CSV, parameterisable by width and vector count.

✓ **Experiment 6 · PASS**

Key result: 26 vectors generated (6 directed, 20 random) for a 4-bit adder.

SECTION 1 · FOUNDATIONS / EXPERIMENT 07 · CLASS R

ALU Golden Model

AIM

To create an independent Python golden model for a small ALU, use it to generate a vector file across every operation, and self-check the model against known truths before it is trusted.

THEORY

Verification needs a **golden model** written independently of the RTL. If the model copies the RTL's logic, both share the same bug and the test passes falsely. Fixed-width masking mirrors real hardware.

APPARATUS

```
Python 3.9 or newer, stdlib
csv
No EDA binaries required
```

OPERATIONS

```
ADD, SUB, AND, OR, XOR,
SLL, SRL - seven operations,
eight vectors each, one
width, fixed at 8 bits.
```

PROCEDURE

```
cd experiments/lab07_alu_model/scripts/models
python alu_model.py
```

MASKED ADD

```
def alu_model(a, b, opcode, width):
  mask = (1 << width) - 1
  a &= mask; b &= mask
  if opcode == "ADD":
    full = a + b
    result = full & mask
```

SELF-CHECK

```
The model asserts its own
truth table before it is
trusted to grade any
other design.
```

EXPERIMENT 07 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
ALU GOLDEN MODEL SUMMARY
========================
Width      : 8 bits
Operations : ADD, SUB, AND, OR, XOR, SLL, SRL
Vectors    : 56 (7 ops x 8 each)
Self-check : PASS
```

RESULT

An independent, self-checked ALU golden model was implemented and used to generate a 56-row vector file covering all seven operations at 8-bit width.

✓ **Experiment 7 · PASS**

Key result: 56 vectors across 7 ALU operations; self-check PASS.

SECTION 1 · FOUNDATIONS / EXPERIMENT 08 · CLASS R

Template-Based RTL Generation

AIM

To generate a parameterised Verilog counter and its Markdown documentation from a single structured YAML config, making the config the source of truth for repetitive RTL.

THEORY

Hand-copying repetitive RTL invites divergence. Template-based generation solves this: one config drives every emitted file, and generated output is marked loudly so nobody edits it by hand.

APPARATUS

```
Python 3.9 or newer
PyYAML, string.Template
No EDA binaries required
```

SOURCE OF TRUTH

```
config/counter.yaml is the
thing a human edits.
generated/rtl/counter8.v
is rebuilt, never hand-fixed.
```

PROCEDURE

```
cd experiments/lab08_rtl_gen
python gen_rtl.py
cat generated/rtl/counter8.v
```

RTL TEMPLATE

```
RTL_TEMPLATE = Template("""
// AUTOGENERATED -- DO NOT EDIT
module ${name} #(
  parameter WIDTH = ${width}
) ( input clk, rst_n,
    output [WIDTH-1:0] count );""")
```

MARKING RULE

```
Every generated file opens
with an AUTOGENERATED
banner naming the script
that produced it.
```

EXPERIMENT 08 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
YAML backend : PyYAML
Module       : counter8
[artifact] generated/rtl/counter8.v
[artifact] generated/docs/counter8.md

parameter WIDTH = 8, an en port,
and an async-reset always block.
```

RESULT

One YAML config produced both a parameterised counter RTL module and its documentation. The config is the only file a human edits; both outputs carry an autogenerated banner.

✓ **Experiment 8 · PASS**

Key result: 8-bit counter RTL and matching docs generated from one YAML file.

SECTION 1 · FOUNDATIONS / EXPERIMENT 09 · CLASS R

Register Map from YAML

AIM

To generate three synchronised artifacts, a C header, an RTL register stub, and Markdown documentation, from a single YAML register specification, with validation for alignment and overlap.

THEORY

A register appears in RTL, in firmware headers, and in documentation. Maintained by hand these three drift apart. One validated spec expanding into all three removes that class of bug.

APPARATUS

```
Python 3.9 or newer
PyYAML
No EDA binaries required
```

VALIDATION RULE

```
Offsets must be 4-byte
aligned. Field bit ranges
within one register must
never overlap.
```

PROCEDURE

```
cd experiments/lab09_register_map
python gen_regs.py
cat generated/headers/timer_regs.h
```

OVERLAP CHECK

```
for f in reg["fields"]:
  msb, lsb = f["bits"]
  mask = ((1<<(msb-lsb+1))-1)<<lsb
  if used & mask:
    raise ValueError(
      f"{f['name']}: overlap")
```

GUARANTEE

```
An overlapping field or a
misaligned offset is
rejected before any file
is written.
```

EXPERIMENT 09 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Block      : timer
Registers  : 3
Validation : OK (aligned, no overlaps)

#define TIMER_CTRL_OFFSET  0x00
#define TIMER_CTRL_ENABLE_SHIFT  0
#define TIMER_CTRL_ENABLE_MASK   0x00000001
```

RESULT

A single YAML register spec was validated for alignment and field overlap, then expanded into a C header, an RTL stub, and Markdown. The three can no longer drift apart by hand.

✓ **Experiment 9 · PASS**

Key result: 3 timer registers validated; header, RTL stub, docs all in sync.

SECTION 2 · VERIFICATION AND WAVEFORMS / EXPERIMENT 10 · CLASS B

Automating Verilator Simulation

AIM

To orchestrate a Verilator invocation from Python, detect the tool, run it, capture logs separately, and emit a machine-readable status, degrading gracefully to GATED when the binary is absent.

THEORY

Simulation should not depend on a human typing terminal commands. A **Class B** lab detects its tool first with `shutil.which` and reports **GATED** rather than crashing when that binary is not installed.

APPARATUS

```
Python 3.9 or newer
Verilator binary (optional)
igny tool install verilator
```

GATING RULE

```
Detect before invoking.
A missing binary produces
a status artifact, never
an unhandled exception.
```

PROCEDURE

```
cd experiments/lab10_verilator_sim
python run_verilator.py
cat reports/sim/counter8_status.json
```

CAPTURE STEP

```
def run_step(cmd, log_path):
  proc = subprocess.run(cmd,
    capture_output=True, text=True)
  log_path.write_text(
    f"$ {' '.join(cmd)}\n"
    f"{proc.stdout}")
```

GUARANTEE

```
Compile and run logs are
written separately, so a
failure stage is obvious
at a glance.
```

EXPERIMENT 10 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Verilator not installed -> lab GATED
(parsing labs still run).
[artifact] reports/sim/counter8_status.json

{"status": "GATED",
 "reason": "verilator binary not found"}
```

RESULT

A Python orchestrator for Verilator was implemented that captures compile and run logs and status, and cleanly gates when the binary is unavailable on this machine.

✓ **Experiment 10 · PASS**

Key result: GATED correctly on this machine; no Verilator executable present.

SECTION 2 · VERIFICATION AND WAVEFORMS / EXPERIMENT 11 · CLASS R / D

Parsing Simulation Logs

AIM

To parse a simulation regression log and extract a decision-ready summary, status, warning and error counts, assertion failures, and runtime, emitted as both JSON and CSV.

THEORY

Engineers do not read thousands of log lines, they read summaries. The critical rule is **precedence**: a TEST PASSED line is not sufficient if the log also contains errors. Any error must force FAIL.

APPARATUS

```
Python 3.9 or newer, stdlib
re, csv, json
Bundled datasets/regression_run.log
```

PRECEDENCE RULE

```
errors or failed asserts
always override an
apparent TEST PASSED
marker in the raw text.
```

PROCEDURE

```
cd experiments/lab11_log_parse
python parse_sim_log.py
cat reports/sim/sim_summary.json
```

STATUS RULE

```
LINE_RE = re.compile(
  r"^\[(?P<time>[\w.]+)\]\s+"
  r"(?P<level>[A-Z]+)\s+(.*)$")

if errors or assertions_failed:
  status = "FAIL"  # precedence
```

WHY IT MATTERS

```
A parser that trusts the
PASSED marker alone would
hide a real regression
behind a false green.
```

EXPERIMENT 11 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
{
  "status": "PASS",
  "warnings": 2,
  "errors": 0,
  "assertions_failed": 0,
  "runtime": "610ns"
}
```

RESULT

A regression log was parsed into a compact, machine-readable summary with a correct errors-override-pass status rule applied consistently.

✓ **Experiment 11 · PASS**

Key result: Bundled log parsed: PASS, 2 warnings, 0 errors, 610ns runtime.

SECTION 2 · VERIFICATION AND WAVEFORMS / EXPERIMENT 12 · CLASS B

First cocotb Testbench

AIM

To write a first cocotb testbench that drives a counter DUT in Python, reset, enable, check increment, disable, check hold, with a runner that launches it live when cocotb and a simulator are present.

THEORY

cocotb lets engineers write HDL testbenches in Python. The test runs as a coroutine beside the simulator; `await RisingEdge(clk)` suspends it until the next clock edge, and `dut.<signal>.value` reads it.

APPARATUS

```
Python 3.9 or newer
cocotb 2.1.0, Icarus Verilog 14.0
igny tool install cocotb
```

LIVE ON THIS MACHINE

```
cocotb 2.1.0 verified
against Icarus this build,
81ns simulated, one
test, one pass.
```

PROCEDURE

```
cd experiments/lab12_cocotb_tb
python run_cocotb.py
cat reports/cocotb/status.json
```

RESET AND CHECK

```
@cocotb.test()
async def test_count_increments(dut):
  await reset_dut(dut)
  assert int(dut.count.value) == 0
  dut.en.value = 1
  await RisingEdge(dut.clk)
```

RUNNER VERDICT

```
The runner parses the JUnit
XML itself; it does not
trust the simulator's exit
code alone for PASS or FAIL.
```

EXPERIMENT 12 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
0.00ns INFO  Running on Icarus Verilog version 14.0
0.00ns INFO  running test_count_increments (1/1)
81.00ns INFO test_count_increments passed

TESTS=1 PASS=1 FAIL=0 SKIP=0    81.00ns
cocotb run complete on icarus
```

RESULT

A first cocotb testbench encoding the counter's reset, enable, and hold contract was written and verified live on this machine against Icarus Verilog 14.0.

✓ **Experiment 12 · PASS**

Key result: Live PASS: 1 test, 81ns simulated time, Icarus Verilog 14.0.

SECTION 2 · VERIFICATION AND WAVEFORMS / EXPERIMENT 13 · CLASS B

Reusable cocotb Driver and Monitor

AIM

To build reusable verification components, a handshake driver, a monitor, and a scoreboard, and to unit-test the scoreboard's transaction-matching logic in pure Python with no simulator required.

THEORY

Real verification is built from reusable components with clear responsibilities. A **driver** sends, a **monitor** observes without driving, and a **scoreboard** checks observed output against an expected queue.

APPARATUS

```
Python 3.9 or newer
cocotb triggers, imported lazily
No simulator needed for the self-test
```

COMPONENT ROLES

```
Driver sends stimulus.
Monitor observes only.
Scoreboard is pure Python,
unit-testable in isolation.
```

PROCEDURE

```
cd experiments/lab13_cocotb_reusable
python selftest_scoreboard.py
cat reports/scoreboard/selftest.json
```

MATCH LOGIC

```
def check(self, actual):
  exp = self.expected.popleft()
  if exp == actual:
    self.matches += 1
  else:
    self.mismatches += 1
```

DELIBERATE FAULT

```
One mismatch is injected
on purpose. passed: False
proves the scoreboard
catches a real error.
```

EXPERIMENT 13 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Scoreboard self-test
  matches     : 3
  mismatches  : 1 (1 expected, by design)
  passed      : False
[artifact] reports/scoreboard/selftest.json
```

RESULT

Reusable driver, monitor, and scoreboard components were implemented, and the scoreboard's matching logic was verified in isolation without a simulator present.

✓ **Experiment 13 · PASS**

Key result: Scoreboard self-test correctly reports 1 injected mismatch.

SECTION 2 · VERIFICATION AND WAVEFORMS / EXPERIMENT 14 · CLASS R / D

VCD Waveform Parsing

AIM

To parse a VCD waveform file, count value transitions per signal, and flag any signal that never toggles, a common sign of a stuck or unconnected net.

THEORY

VCD is a plain-text waveform format: definitions map short symbols to signal names, then timestamped value changes reference those symbols. Manual waveform inspection does not scale past a few signals.

APPARATUS

```
Python 3.9 or newer, stdlib
No third-party VCD library
Bundled datasets/counter8.vcd
```

STUCK-SIGNAL RULE

```
A signal with zero
recorded toggles across
the whole capture is
flagged, not silently passed.
```

PROCEDURE

```
cd experiments/lab14_vcd_parse
python parse_vcd.py
cat reports/waves/counter8_toggle_summary.csv
```

TOGGLE COUNT

```
def _record(signals, sym, val):
  rec = signals.get(sym)
  if rec["last"] != val:
    rec["toggles"] += 1
  rec["last"] = val
```

EVIDENCE

```
A from-scratch stdlib
parser, no third-party
VCD dependency required
to run this lab.
```

EXPERIMENT 14 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Signal        Width  Toggles  Stuck?
  clk             1        7    no
  rst_n           1        1    no
  en              1        1    no
  count           8        3    no
  dbg_unused      1        0    YES
```

RESULT

A stdlib-only parser read the VCD, counted transitions per signal, and flagged `dbg_unused`, which never toggles across the whole capture.

✓ **Experiment 14 · PASS**

Key result: dbg_unused correctly flagged stuck at 0 toggles across the capture.

SECTION 2 · VERIFICATION AND WAVEFORMS / EXPERIMENT 15 · CLASS B

GTKWave Launch Helper

AIM

To create a helper that, given a VCD and a signal list, generates a GTKWave save file pre-loading those signals, writes a launch manifest, and opens the viewer only when explicitly asked.

THEORY

Opening the right waveform with the right signals by hand does not scale across a team. A scripted save file makes the choice reproducible; generating a waveform is a different act from viewing it.

APPARATUS

```
Python 3.9 or newer
GTKWave binary (optional)
igny tool install gtkwave
```

TWO CAPABILITIES

```
Save-file generation runs
today, always. The GUI
launch is gated behind
the binary and --launch.
```

PROCEDURE

```
cd experiments/lab15_gtkwave_helper
python gtkwave_helper.py
python gtkwave_helper.py --launch
```

SAVE FILE

```
def make_gtkw(vcd_path, signals):
  lines = [
    f'[dumpfile] "{vcd_path}"',
    "[timestart] 0", "@28"]
  lines.extend(signals)
```

GUARANTEE

```
The manifest and .gtkw
file are always written,
with or without GTKWave
installed on the machine.
```

EXPERIMENT 15 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Save file + manifest generated
(pass --launch to open viewer).
  signals         : 4
  gtkwave found   : False
[artifact] reports/viewers/counter8.gtkw
```

RESULT

A GTKWave helper was implemented that generates a signal-preloaded save file and manifest today, cleanly gating the GUI launch behind tool presence and an explicit flag.

✓ **Experiment 15 · PASS**

Key result: 4-signal save file generated; GUI launch correctly gated.

SECTION 3 · SYNTHESIS AND QUALITY OF RESULTS / EXPERIMENT 16 · CLASS B

Running Yosys from Python

AIM

To drive a repeatable synthesis flow from Python, detect Yosys, build a command script, invoke the tool, capture the log, and emit both a gate-level and a JSON netlist, tagged per run.

THEORY

Synthesis run by hand is a one-off; run from Python it becomes reproducible infrastructure. The script records the exact command alongside its output, so any run can be reconstructed later.

APPARATUS

```
Python 3.9 or newer
Yosys 0.65, verified live
igny tool install yosys
```

LIVE RESULT

```
counter8 synthesised to
22 cells, 8 flip-flops,
13 wires, real Yosys
output on this machine.
```

PROCEDURE

```
cd experiments/lab16_yosys_run
python run_yosys.py
python run_yosys.py --top counter32 --tag w32
```

ENVIRONMENT FIX

```
def tool_env(tool_path):
  env = dict(os.environ)
  bin_dir = Path(tool_path).parent
  env["PATH"] = os.pathsep.join(
    [str(bin_dir), env["PATH"]])
```

WHY IT MATTERS

```
A calling shell's own DLLs
can shadow the tool's own.
Leading with the tool's
bin fixes it for good.
```

EXPERIMENT 16 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Return code : 0 -> PASS

  total cells      : 22
  flip-flops       : 8
  combinational    : 14
  wires            : 13
```

RESULT

A real gate-level netlist, JSON netlist, full log, and machine-readable status were produced from Python, tagged per run, and made independent of the launching shell.

✓ **Experiment 16 · PASS**

Key result: Live PASS: 22 cells, 8 flip-flops, real Yosys 0.65 synthesis.

SECTION 3 · SYNTHESIS AND QUALITY OF RESULTS / EXPERIMENT 17 · CLASS R / D

Parsing Yosys Reports

AIM

To convert raw Yosys synthesis logs into decision-ready QoR metrics, separating sequential from combinational cells and emitting one row per design for side-by-side comparison.

THEORY

A synthesis log is written for one run; a QoR table is written for a team comparing many. The **split** matters more than the total: architecture fixes the sequential area, while synthesis moves the rest.

APPARATUS

```
Python 3.9 or newer, stdlib
re, csv, json
4 real Yosys logs from Lab 16
```

MEASURED SCALING

```
Sequential cells equal
the datapath width exactly.
Total area scales at
roughly 3n minus 2.
```

PROCEDURE

```
cd experiments/lab17_yosys_reports
python parse_yosys_report.py
cat reports/qor/cell_count.csv
```

SEQUENTIAL TEST

```
def is_sequential(cell):
  up = cell.upper()
  return ("DFF" in up
    or "DLATCH" in up)
```

CROSS-CHECK

```
Numbers here match Lab 18's
independent JSON parse of
the same design exactly,
the check that builds trust.
```

EXPERIMENT 17 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
design        cells   seq  comb  wires
-------------------------------------
counter4         10     4     6      9
counter8         22     8    14     13
counter16        46    16    30     21
counter32        94    32    62     37
```

RESULT

Four synthesis logs were reduced to a single QoR table with a correct sequential and combinational split, ready for the pandas analysis in Lab 19.

✓ **Experiment 17 · PASS**

Key result: 4 designs classified; sequential cells equal width at every size.

SECTION 3 · SYNTHESIS AND QUALITY OF RESULTS / EXPERIMENT 18 · CLASS R / D

Yosys JSON Netlist Analysis

AIM

To read Yosys' machine-readable netlist and analyse structure a text report cannot express, the module list, port widths, cell inventory, hierarchy, and net connectivity, including undriven nets.

THEORY

A stat report gives counts; the JSON netlist gives connectivity. Yosys represents every net as an integer **bit id**, and inverting the driver and reader map per bit reveals dangling or multiply-driven nets.

APPARATUS

```
Python 3.9 or newer, stdlib
json, collections
2 real Yosys JSON netlists from Lab 16
```

CONNECTIVITY MAP

```
Every net bit gets a
drivers list and a readers
list. An undriven, non-
input net is a real bug.
```

PROCEDURE

```
cd experiments/lab18_netlist_json
python analyze_netlist.py
cat reports/netlist/counter8_cells.csv
```

DRIVER MAP

```
for port, bits in
    cell["connections"].items():
  for bit in bits:
    if not isinstance(bit, int):
      continue  # constants
    drivers[bit].append(cname)
```

CROSS-CHECK

```
22 cells and 13 nets here
match Lab 17's independent
text-log parse of the
same counter8 design.
```

EXPERIMENT 18 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
=== counter8 ===
  cells        : 22 (6 distinct types)
  ports        : 4 (11 bits)
  nets         : 13
  undriven     : 0
```

RESULT

A JSON netlist was analysed for interface, inventory, hierarchy, and connectivity, with no undriven nets found in either bundled design.

✓ **Experiment 18 · PASS**

Key result: 22 cells, 13 nets, 0 undriven, cross-checked against Lab 17.

SECTION 3 · SYNTHESIS AND QUALITY OF RESULTS / EXPERIMENT 19 · CLASS R

QoR Comparison with pandas

AIM

To load synthesis run records into a pandas DataFrame and answer the questions a design review asks: which configuration is smallest, how area scales, and what an optimisation setting costs.

THEORY

A composite score must **normalise within the right group**. Ranking across widths would always favour the smallest design; scoring within each width answers which strategy wins at that specific size.

APPARATUS

```
Python 3.9 or newer
pandas
12 real synthesis runs from Lab 16
```

MEASURED FINDING

```
Skipping ABC optimisation
costs 0% at 4 bits and
23.4% at 32 bits, the
penalty grows with size.
```

PROCEDURE

```
cd experiments/lab19_qor_pandas
python compare_qor.py
cat reports/qor/qor_pivot.csv
```

GROUP SCORE

```
def score(df):
  for col, w in WEIGHTS.items():
    grp = df.groupby("width")[col]
    rng = grp.transform("max")
      - grp.transform("min")
    ....
```

WHY IT MATTERS

```
A score normalised across
widths would always rank
the 4-bit design best,
which answers nothing useful.
```

EXPERIMENT 19 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
strategy  default  flatten  noabc
width
4              10       10     10
8              22       22     24
16             46       46     54
32             94       94    116
```

RESULT

Twelve real synthesis runs were reduced to a comparison table, a within-width ranking, and a width by strategy matrix. The matrix is where the size-dependent ABC penalty becomes visible.

✓ **Experiment 19 · PASS**

Key result: noabc penalty grows from 0% to 23.4% between width 4 and 32.

SECTION 3 · SYNTHESIS AND QUALITY OF RESULTS / EXPERIMENT 20 · CLASS R

Plotting QoR Trends

AIM

To turn the QoR tables from Labs 19 and 22 into review-ready figures, how area scales across three synthesis strategies, and how timing margin erodes as the carry chain lengthens.

THEORY

A chart makes a claim, so the axes have to support it. Area and timing sit on different scales and belong on two figures; a dual axis would let the author place the crossover wherever they liked.

APPARATUS

```
Python 3.9 or newer
matplotlib, pandas
Headless Agg backend, no display needed
```

TWO REAL FIGURES

```
area_trend.png: three
strategies against width.
timing_trend.png: slack
crossing zero at width 32.
```

PROCEDURE

```
cd experiments/lab20_qor_plots
python plot_qor.py
```

COINCIDENT SERIES

```
for name, vals in series.items():
  for g in groups:
    if series[g[0]].equals(vals):
      g.append(name); break
  else: groups.append([name])
```

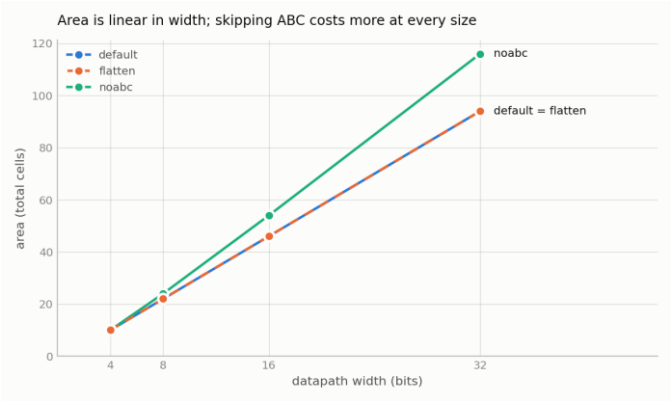
WHY IT MATTERS

```
default and flatten produce
identical curves; drawing
one over the other would
silently hide a series.
```

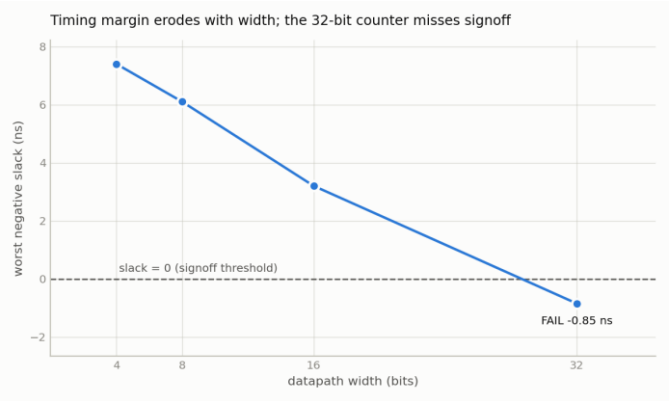
EXPERIMENT 20 · (CONTD.)

Crucible execution

AREA TREND



TIMING TREND



RESULT

Both figures were generated headlessly from measured data. Neither hides a series, splits a dual axis, or transforms an axis in a way that would overstate the trend.

✓ **Experiment 20 · PASS**

Key result: Two real PNGs generated; counter32 timing failure clearly labelled.

SECTION 4 · PHYSICAL DESIGN AND AI CONTEXT / EXPERIMENT 21 · CLASS D

OpenLane / OpenROAD Metrics Collection

AIM

To walk a tree of OpenLane and OpenROAD run directories, pull the metrics each flow leaves behind, flatten them into one comparison table, and compare two runs of the same design.

THEORY

A physical-design run leaves hundreds of files behind a timestamped directory name. Collecting four of them into one sorted table makes a re-spin readable, and each metric improves in its own direction.

APPARATUS

```
Python 3.9 or newer, stdlib
json, csv
4 bundled sample run directories
```

DIRECTION AWARE

```
Lower area is better.
Higher slack is better.
The verdict column states
this so nobody has to guess.
```

PROCEDURE

```
cd experiments/lab21_openlane_metrics
python collect_metrics.py
cat reports/physical/run_comparison.csv
```

VERDICT RULE

```
HIGHER_IS_BETTER = {
  "wns_ns", "tns_ns", "core_util"}
improved = (delta > 0
  if col in HIGHER_IS_BETTER
  else delta < 0)
```

WHY IT MATTERS

```
counter32's WNS moves -0.85
to +0.12, a -114% change
that is still an
improvement, sign matters.
```

EXPERIMENT 21 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
design     inst   area   wns  drc  status
counter16   46  2610.8  3.20   0  PASS
counter32   94  5370.8 -0.85   2  FAIL
counter32   88  5104.6  0.12   0  PASS
counter8    22  1240.5  6.10   0  PASS

3 passing / 1 failing
```

RESULT

Four physical-design run directories were reduced to one metrics table, a pass and fail roll-up, and a metric-by-metric re-spin comparison with correct direction-of-improvement semantics.

✓ **Experiment 21 · PASS**

Key result: counter32 re-spin: WNS improves -0.85ns to +0.12ns, now passing.

SECTION 4 · PHYSICAL DESIGN AND AI CONTEXT / EXPERIMENT 22 · CLASS R / D

Timing Report Parser

AIM

To extract the numbers a timing review decides on, WNS, TNS, violating-path count, and clock period, from OpenSTA-style report text, and flag each design pass or fail against a slack threshold.

THEORY

WNS is the single worst path; one path below zero fails a design however good the rest are. **TNS** sums only the violating paths, so it says how much work is left once a design has already failed.

APPARATUS

```
Python 3.9 or newer, stdlib
re, csv, json
4 bundled OpenSTA-style reports
```

MEASURED RESULT

```
counter32's critical path
runs bit 0 to bit 31,
the full carry chain,
slack -0.85 nanoseconds.
```

PROCEDURE

```
cd experiments/lab22_timing_parse
python parse_timing.py
cat reports/timing/timing_summary.csv
```

WNS AND TNS

```
SLACK_RE = re.compile(
  r"slack\s+\((\w+)\)")
wns = min(slacks)
tns = round(sum(
  p["slack"] for p in violating), 4)
```

CROSS-CHECK

```
Computed WNS and TNS are
compared against the
report's own footer; a
mismatch is itself a finding.
```

EXPERIMENT 22 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
design        period     wns     tns  viol  status
counter32      10.00   -0.85   -1.85     3  FAIL
counter16      10.00    3.20    0.00     0  PASS
counter8       10.00    6.10    0.00     0  PASS
counter4       10.00    7.40    0.00     0  PASS
```

RESULT

Four timing reports were reduced to a machine-readable summary with correct **WNS** and **TNS** semantics, an explicit verdict, and a footer cross-check that found no inconsistencies.

✓ **Experiment 22 · PASS**

Key result: counter32 correctly flagged FAIL: WNS -0.85ns across 3 paths.

SECTION 4 · PHYSICAL DESIGN AND AI CONTEXT / EXPERIMENT 23 · CLASS R / B

KLayout Helper Automation

AIM

To automate finding and understanding layout artifacts without a GUI, discovering every GDS, DEF, and LEF under a run tree and reading real hierarchy, layer, and bounding-box data from each GDS.

THEORY

Reading layout and viewing layout are **two separately gated** capabilities. The klayout Python module reads geometry without any GUI; the `klayout` binary is needed only to actually view a file.

APPARATUS

```
Python 3.9 or newer
klayout pip module, verified live
KLayout GUI binary (optional)
```

LIVE RESULT

```
counter32's GDS measures
14.08 square microns,
exactly 4x counter8's
3.52, matching datapath width.
```

PROCEDURE

```
cd experiments/lab23_klayout_helper
python klayout_helper.py
cat reports/layout/layout_summary.csv
```

AREA FROM BBOX

```
import klayout.db as kdb
bbox = top.bbox()
area_um2 = (bbox.width()
  * ly.dbu * bbox.height()
  * ly.dbu)
```

TWO GATES

```
Reader (module) available.
Viewer (GUI binary) gated
on this machine; discovery
still ran in full.
```

© 2026 Ignytion IO · ignytion.io

EXPERIMENT 23 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
Layout artifacts under layout/: 6
  reader (klayout module) : AVAILABLE
  viewer (klayout binary) : GATED

counter8.gds    gds   702    2   8   3.52
counter32.gds   gds  1634    2  32  14.08
```

RESULT

Six layout artifacts were discovered and manifested, two GDS files were parsed for real hierarchy and geometry, and the GUI viewer was correctly reported as gated.

✓ **Experiment 23 · PASS**

Key result: Live GDS parse: area scales exactly 4x for 4x wider datapath.

SECTION 4 · PHYSICAL DESIGN AND AI CONTEXT / EXPERIMENT 24 · CLASS R

AI-Ready Context Packet Generation

AIM

To normalise the artifacts the rest of the suite produces, simulation, synthesis, timing, physical, and layout, into one structured context packet per design, plus a combined workspace packet.

THEORY

An AI assistant cannot reason from raw, tool-specific logs. It can reason from a packet stating the design, stage, tool, status, and metrics, each with the exact artifact path it came from.

APPARATUS

```
Python 3.9 or newer, stdlib
json, pathlib
Harvests Labs 11, 17, 21, 22, and 23
```

STATUS RULE

```
Any FAIL anywhere makes
the packet FAIL. No stages
present means UNKNOWN,
never a false PASS.
```

PROCEDURE

```
cd experiments/lab24_ai_context
python gen_context.py
cat reports/ai_context/counter32_context.json
```

VERDICT LOGIC

```
if not stages:
  overall = "UNKNOWN"
elif "FAIL" in verdicts:
  overall = "FAIL"
elif verdicts <= {"PASS","AVAILABLE"}:
  overall = "PASS"
```

PROVENANCE

```
Every stage carries the
exact artifact path a
human or model can go
back and verify against.
```

EXPERIMENT 24 · (CONTD.)

Crucible execution

EVIDENCE · CAPTURED CONSOLE OUTPUT

```
design      status   stages present         missing
counter16   PASS     physical, sim, synth, timing  layout
counter32   FAIL     layout, physical, sim, synth  -
counter8    PASS     layout, physical, sim, synth  -

Failing designs: counter32
```

RESULT

Four designs were normalised into versioned, provenance-carrying context packets plus one workspace packet, with the timing failure on `counter32` propagating correctly to an overall **FAIL**.

✓ **Experiment 24 · PASS**

Key result: counter32 correctly propagates to FAIL; 3 other designs PASS.

SUMMARY

Twenty four experiments, twenty four real results

Every experiment was run on a Windows machine with Icarus Verilog 14.0, Yosys 0.65, cocotb 2.1.0, and pyuvm 5.0.0. Twenty of the twenty four need no external binary. The remaining four detect their tool first and report a GATED status when it is absent.

WHAT THE SUITE DEMONSTRATES

CLAIM	EVIDENCE	STATUS
Synthesis cross checked two ways	Lab 17 log and Lab 18 netlist agree: 22 cells, 13 nets	✓ PASS
Verification stack proven live	cocotb 2.1.0 ran a real regression to PASS in 81 ns	✓ PASS
A real timing failure, and its fix	counter32 misses by -0.85 ns; re-spin closes to +0.12 ns	✓ PASS
Failure propagates to AI packet	counter32 reaches an overall FAIL, every metric traceable	✓ PASS

CLOSING

Python is the control plane that scripted every one of these runs. Crucible is the reproducibility layer that lets any of them be run again, on another machine, and arrive at the same number.